SQL

PM566 - Week 9

Emil Hvitfeldt, George G. Vega Yon, Kelly Street

Acknowledgment

These slides were originally developed by Emil Hvitfeldt.

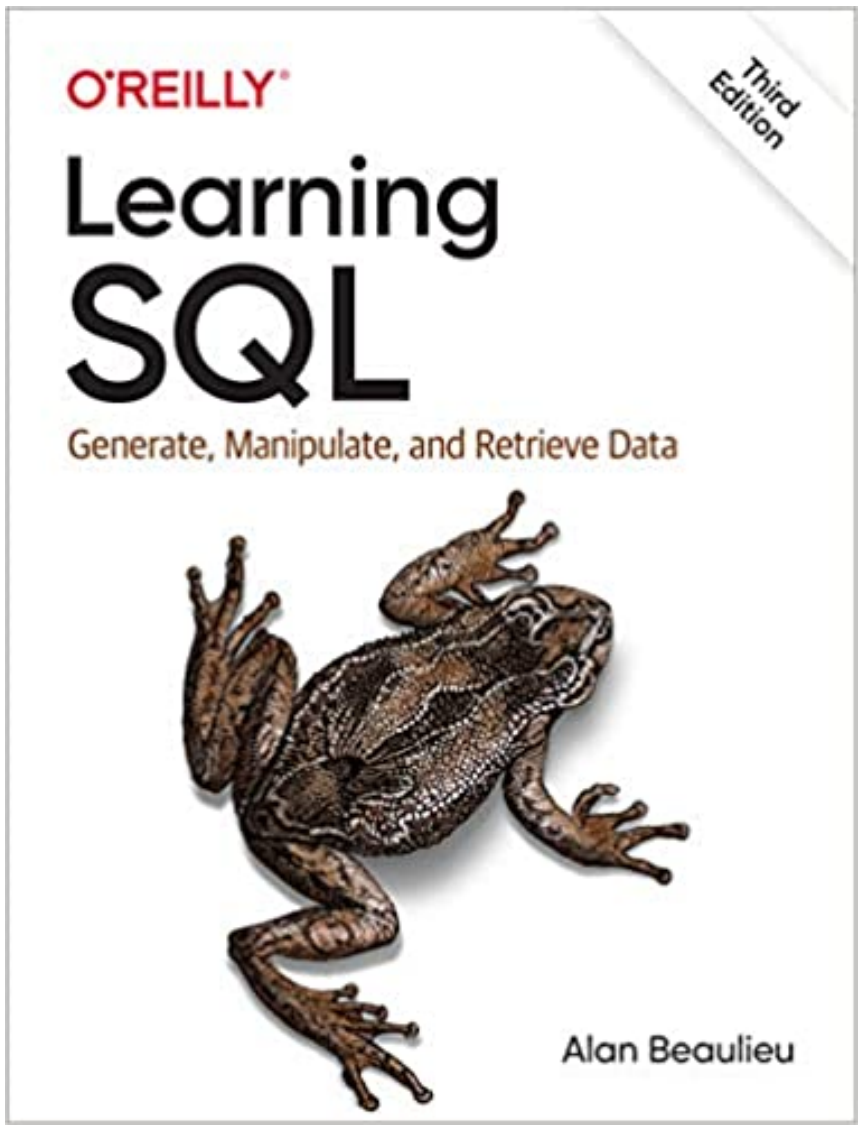
Plan for the week

- See what relational databases are
- How to query a relational database with SQL
- Similarities between SQL and dplyr

NOTE

SQL is a full language with 100's of commands. We will not be able to cover everything. We are going to look at a small sample to showcase the powers, capabilities and similarities to `dplyr`.

Optional reading material



Database

Simple definition

> A set of related information

Examples include:

- phone book
- Dictionary
- Cookbook

Order / indexing matters

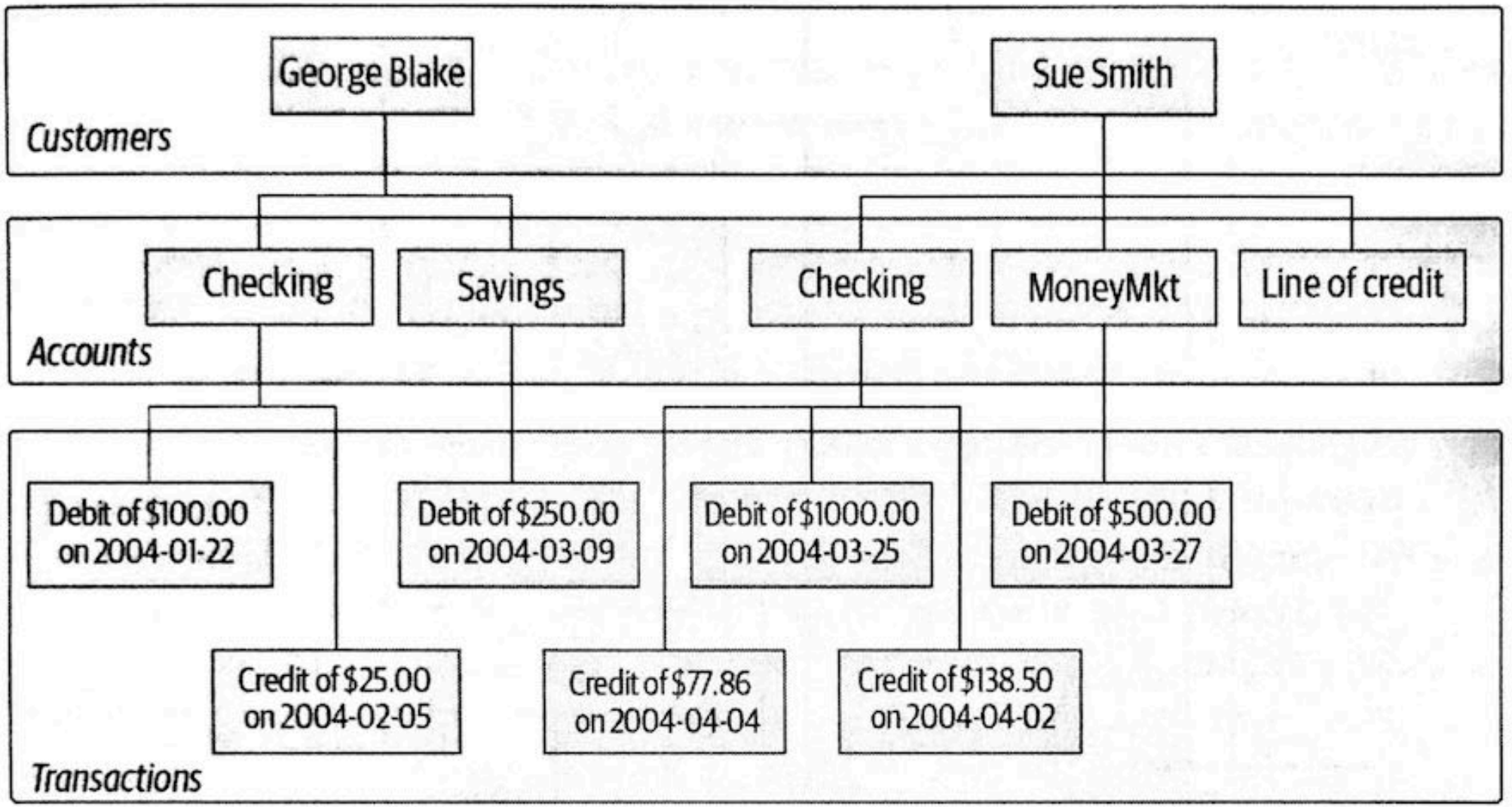
The ordering of the entries will make some tasks easier and some harder.

A phone book makes it easy to find a person if you know the name, but not if you only know the address

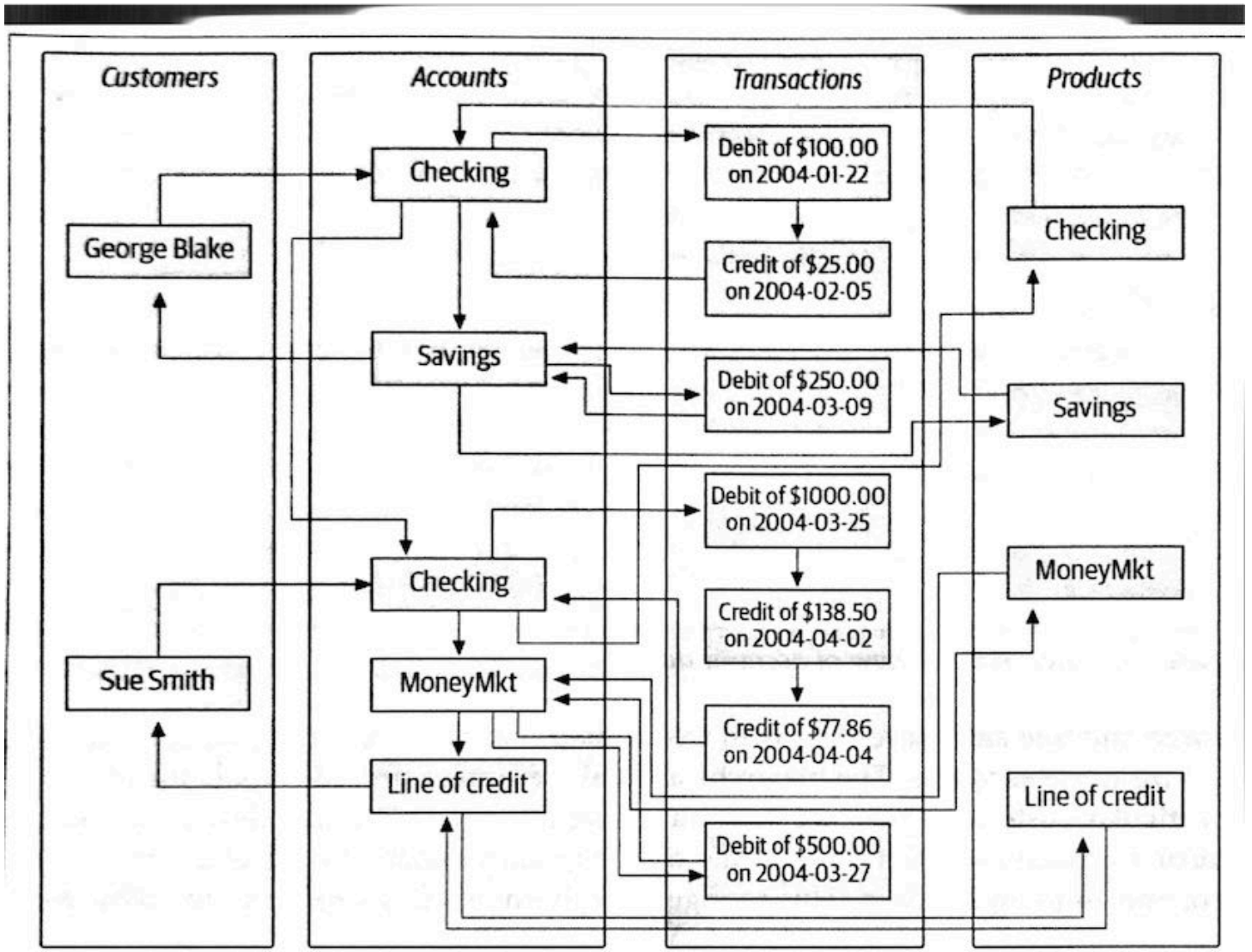
Dictionaries are not helpful to find the longest words

Once the book is printed then it starts going out of date

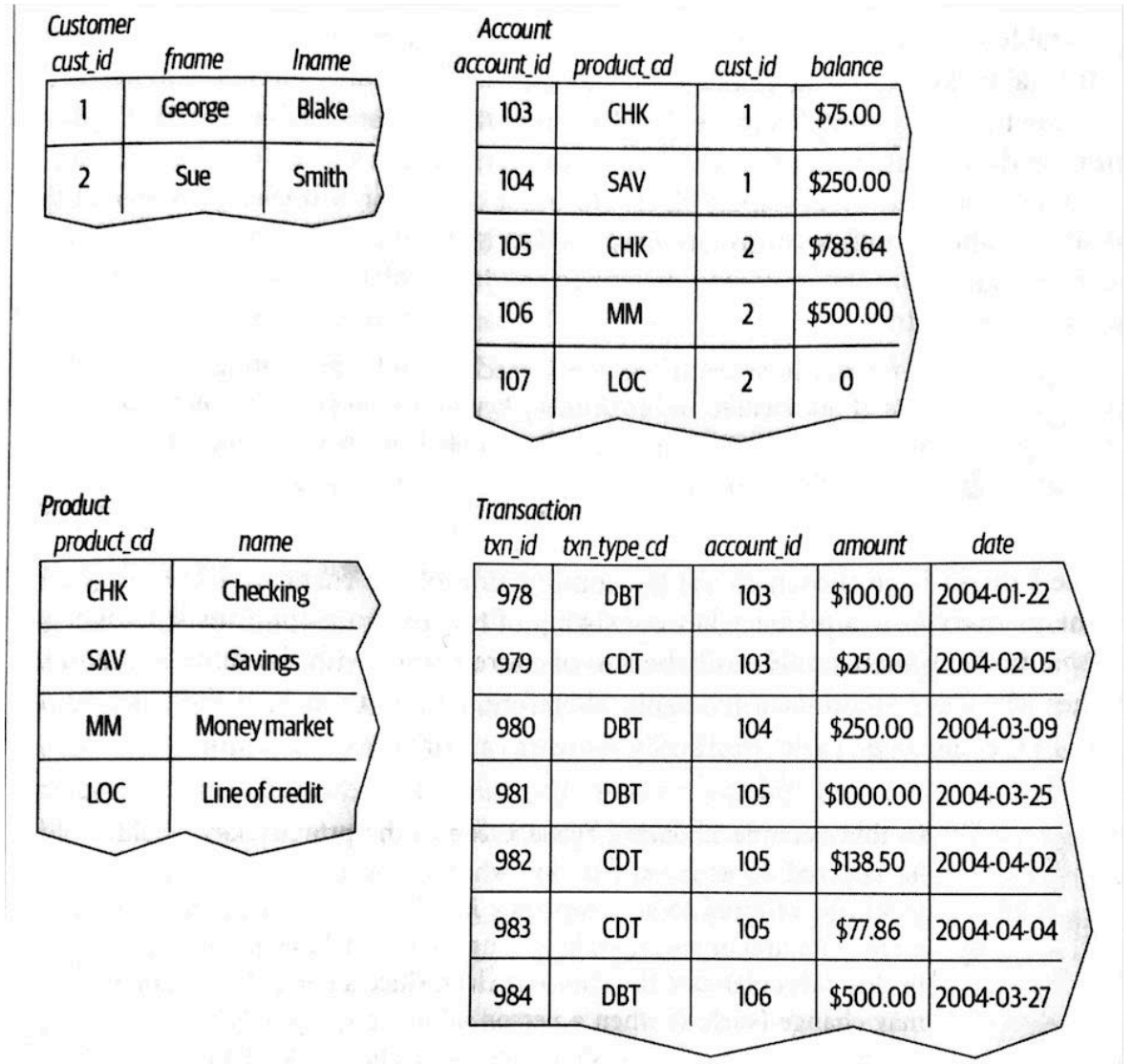
Hierarchical database system



Network database system



Relational database system



Customer

cust_id	*fname*	*lname*
1	George	Blake
2	Sue	Smith

Account

account_id	*product_cd*	*cust_id*	*balance*
103	CHK	1	$75.00
104	SAV	1	$250.00
105	CHK	2	$783.64
106	MM	2	$500.00
107	LOC	2	0

Product

product_cd	*name*
CHK	Checking
SAV	Savings
MM	Money market
LOC	Line of credit

Transaction

txn_id	*txn_type_cd*	*account_id*	*amount*	*date*
978	DBT	103	$100.00	2004-01-22
979	CDT	103	$25.00	2004-02-05
980	DBT	104	$250.00	2004-03-09
981	DBT	105	$1000.00	2004-03-25
982	CDT	105	$138.50	2004-04-02
983	CDT	105	$77.86	2004-04-04
984	DBT	106	$500.00	2004-03-27

All of these systems contain 4 entities

- `customer`
- `product`
- `account`
- `transaction`

Each table in a relational database contains information that uniquely identifies that row in the table (primary key).

Relation between tables

Some columns contain shared information across tables.

These columns are called *foreign keys* and they serve the purpose of connecting the rows in different tables together.

Terminology

- **Entity** - Something of interest to the users
- **Column** - An individual piece of data stored in a table
- **Row** - A set of columns that together completely describe an entity or some action on an entity. Also called a **record**
- **Table** - A set of rows
- **Result key** - Another name for a non-persistent table, generally the result of a SQL query
- **Primary key** - One or more columns that can be used as a unique identifier for each row in a table
- **Foreign key** - One or more columns that can be used together to identify a single row in another table

What is SQL?

A language created to manipulate data in relational databases.

Over 40 years old.

The American National Standards Institute (ANSI) have developed standards for SQL.

A Nonprocedural Language

In R we are used to being able to define variables, use conditional logic (if, if-else, while), looping/iteration and writing functions.

In general are we in complete control of what the code is doing.

A Nonprocedural Language (cont.)

We give up some of that when using SQL.

When using SQL then we specify the input and output we desire, the this is sent to your database engine known as the **optimizer**

The optimizer figures out the best way to get from input to output

Different databases

There are different SQL databases that operate slightly different. MySQL, PostgreSQL, MariaDB, SQLite.

We will try to give a general overview.

Google is your friend.

All examples shown with SQLite.

SQL query clauses

- **SELECT** Determines which columns to include the query's result set
- **FROM** Identifies the tables from which to retrieve data and how the tables should be joined
- **WHERE** Filters out unwanted data
- **GROUP BY** Used to group rows together by common column values
- **HAVING** Filters out unwanted groups
- **ORDER BY** Sorts the rows of the final result set by one or more columns

Pengiuns data

```r
head(as.data.frame(palmerpenguins::penguins))
```

```
  species    island bill_length_mm bill_depth_mm flipper_length_mm body_mass_g
1  Adelie Torgersen           39.1          18.7                181        3750
2  Adelie Torgersen           39.5          17.4                186        3800
3  Adelie Torgersen           40.3          18.0                195        3250
4  Adelie Torgersen             NA            NA                 NA          NA
5  Adelie Torgersen           36.7          19.3                193        3450
6  Adelie Torgersen           39.3          20.6                190        3650
     sex year
1   male 2007
2 female 2007
3 female 2007
4   <NA> 2007
5 female 2007
6   male 2007
```

Select clause

```sql
SELECT species, island, bill_length_mm
FROM penguins
```

	species	island	bill_length_mm
1	Adelie	Torgersen	39.1
2	Adelie	Torgersen	39.5
3	Adelie	Torgersen	40.3
4	Adelie	Torgersen	NA
5	Adelie	Torgersen	36.7
6	Adelie	Torgersen	39.3
7	Adelie	Torgersen	38.9
8	Adelie	Torgersen	39.2
9	Adelie	Torgersen	34.1
10	Adelie	Torgersen	42.0
11	Adelie	Torgersen	37.8
12	Adelie	Torgersen	37.8
13	Adelie	Torgersen	41.1

Select clause

```sql
SELECT *
FROM penguins
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm
1	Adelie	Torgersen	39.1	18.7	181
2	Adelie	Torgersen	39.5	17.4	186
3	Adelie	Torgersen	40.3	18.0	195
4	Adelie	Torgersen	NA	NA	NA
5	Adelie	Torgersen	36.7	19.3	193
6	Adelie	Torgersen	39.3	20.6	190
7	Adelie	Torgersen	38.9	17.8	181
8	Adelie	Torgersen	39.2	19.6	195
9	Adelie	Torgersen	34.1	18.1	193
10	Adelie	Torgersen	42.0	20.2	190
11	Adelie	Torgersen	37.8	17.1	186
12	Adelie	Torgersen	37.8	17.3	180
13	Adelie	Torgersen	41.1	17.6	182

Select clause - aliases

```sql
SELECT upper(island) AS island,
  bill_length_mm * bill_depth_mm AS bill_area_mm2
FROM penguins
```

	island	bill_area_mm2
1	TORGERSEN	731.17
2	TORGERSEN	687.30
3	TORGERSEN	725.40
4	TORGERSEN	NA
5	TORGERSEN	708.31
6	TORGERSEN	809.58
7	TORGERSEN	692.42
8	TORGERSEN	768.32
9	TORGERSEN	617.21
10	TORGERSEN	848.40
11	TORGERSEN	646.38
12	TORGERSEN	653.94
13	TORGERSEN	723.36

Select clause - removing duplicates

```sql
SELECT DISTINCT island, species
FROM penguins
```

	island	species
1	Torgersen	Adelie
2	Biscoe	Adelie
3	Dream	Adelie
4	Biscoe	Gentoo
5	Dream	Chinstrap

From clause - subquery

```sql
SELECT a.bill_length_mm, a.bill_depth_mm
FROM
  (SELECT species, bill_length_mm, bill_depth_mm
   FROM penguins
   WHERE species = 'Adelie'
  ) AS a
```

	bill_length_mm	bill_depth_mm
1	39.1	18.7
2	39.5	17.4
3	40.3	18.0
4	NA	NA
5	36.7	19.3
6	39.3	20.6
7	38.9	17.8
8	39.2	19.6
9	34.1	18.1
10	42.0	20.2
11	37.8	17.1
12	37.8	17.3
13	41.1	17.6

Where clause

```sql
SELECT species, island, bill_length_mm, bill_length_mm, year
FROM penguins
WHERE year = 2007
```

	species	island	bill_length_mm	bill_length_mm	year
1	Adelie	Torgersen	39.1	39.1	2007
2	Adelie	Torgersen	39.5	39.5	2007
3	Adelie	Torgersen	40.3	40.3	2007
4	Adelie	Torgersen	NA	NA	2007
5	Adelie	Torgersen	36.7	36.7	2007
6	Adelie	Torgersen	39.3	39.3	2007
7	Adelie	Torgersen	38.9	38.9	2007
8	Adelie	Torgersen	39.2	39.2	2007
9	Adelie	Torgersen	34.1	34.1	2007
10	Adelie	Torgersen	42.0	42.0	2007
11	Adelie	Torgersen	37.8	37.8	2007
12	Adelie	Torgersen	37.8	37.8	2007
13	Adelie	Torgersen	41.1	41.1	2007

Where clause

```sql
SELECT species, island, bill_length_mm, bill_length_mm, year
FROM penguins
WHERE year > 2007
```

	species	island	bill_length_mm	bill_length_mm	year
1	Adelie	Biscoe	39.6	39.6	2008
2	Adelie	Biscoe	40.1	40.1	2008
3	Adelie	Biscoe	35.0	35.0	2008
4	Adelie	Biscoe	42.0	42.0	2008
5	Adelie	Biscoe	34.5	34.5	2008
6	Adelie	Biscoe	41.4	41.4	2008
7	Adelie	Biscoe	39.0	39.0	2008
8	Adelie	Biscoe	40.6	40.6	2008
9	Adelie	Biscoe	36.5	36.5	2008
10	Adelie	Biscoe	37.6	37.6	2008
11	Adelie	Biscoe	35.7	35.7	2008
12	Adelie	Biscoe	41.3	41.3	2008
13	Adelie	Biscoe	37.6	37.6	2008

Where clause

```sql
SELECT species, island, bill_length_mm, bill_length_mm, year
FROM penguins
WHERE year IN (2007, 2008)
```

	species	island	bill_length_mm	bill_length_mm	year
1	Adelie	Torgersen	39.1	39.1	2007
2	Adelie	Torgersen	39.5	39.5	2007
3	Adelie	Torgersen	40.3	40.3	2007
4	Adelie	Torgersen	NA	NA	2007
5	Adelie	Torgersen	36.7	36.7	2007
6	Adelie	Torgersen	39.3	39.3	2007
7	Adelie	Torgersen	38.9	38.9	2007
8	Adelie	Torgersen	39.2	39.2	2007
9	Adelie	Torgersen	34.1	34.1	2007
10	Adelie	Torgersen	42.0	42.0	2007
11	Adelie	Torgersen	37.8	37.8	2007
12	Adelie	Torgersen	37.8	37.8	2007
13	Adelie	Torgersen	41.1	41.1	2007

Where clause

```sql
SELECT species, island, bill_length_mm, bill_length_mm, year
FROM penguins
WHERE year = 2007 AND species = 'Gentoo'
```

	species	island	bill_length_mm	bill_length_mm	year
1	Gentoo	Biscoe	46.1	46.1	2007
2	Gentoo	Biscoe	50.0	50.0	2007
3	Gentoo	Biscoe	48.7	48.7	2007
4	Gentoo	Biscoe	50.0	50.0	2007
5	Gentoo	Biscoe	47.6	47.6	2007
6	Gentoo	Biscoe	46.5	46.5	2007
7	Gentoo	Biscoe	45.4	45.4	2007
8	Gentoo	Biscoe	46.7	46.7	2007
9	Gentoo	Biscoe	43.3	43.3	2007
10	Gentoo	Biscoe	46.8	46.8	2007
11	Gentoo	Biscoe	40.9	40.9	2007
12	Gentoo	Biscoe	49.0	49.0	2007
13	Gentoo	Biscoe	45.5	45.5	2007

Where clause

```sql
SELECT species, island, bill_length_mm, bill_length_mm, year
FROM penguins
WHERE year = 2007 OR species = 'Gentoo'
```

	species	island	bill_length_mm	bill_length_mm	year
1	Adelie	Torgersen	39.1	39.1	2007
2	Adelie	Torgersen	39.5	39.5	2007
3	Adelie	Torgersen	40.3	40.3	2007
4	Adelie	Torgersen	NA	NA	2007
5	Adelie	Torgersen	36.7	36.7	2007
6	Adelie	Torgersen	39.3	39.3	2007
7	Adelie	Torgersen	38.9	38.9	2007
8	Adelie	Torgersen	39.2	39.2	2007
9	Adelie	Torgersen	34.1	34.1	2007
10	Adelie	Torgersen	42.0	42.0	2007
11	Adelie	Torgersen	37.8	37.8	2007
12	Adelie	Torgersen	37.8	37.8	2007
13	Adelie	Torgersen	41.1	41.1	2007

Order by clause

```sql
SELECT species, island, bill_length_mm, bill_depth_mm
FROM penguins
ORDER BY bill_length_mm
```

	species	island	bill_length_mm	bill_depth_mm
1	Adelie	Torgersen	NA	NA
2	Gentoo	Biscoe	NA	NA
3	Adelie	Dream	32.1	15.5
4	Adelie	Dream	33.1	16.1
5	Adelie	Torgersen	33.5	19.0
6	Adelie	Dream	34.0	17.1
7	Adelie	Torgersen	34.1	18.1
8	Adelie	Torgersen	34.4	18.4
9	Adelie	Biscoe	34.5	18.1
10	Adelie	Torgersen	34.6	21.1
11	Adelie	Torgersen	34.6	17.2
12	Adelie	Biscoe	35.0	17.9
13	Adelie	Biscoe	35.0	17.9

Order by clause

```sql
SELECT species, island, bill_length_mm, bill_depth_mm
FROM penguins
ORDER BY bill_length_mm DESC
```

	species	island	bill_length_mm	bill_depth_mm
1	Gentoo	Biscoe	59.6	17.0
2	Chinstrap	Dream	58.0	17.8
3	Gentoo	Biscoe	55.9	17.0
4	Chinstrap	Dream	55.8	19.8
5	Gentoo	Biscoe	55.1	16.0
6	Gentoo	Biscoe	54.3	15.7
7	Chinstrap	Dream	54.2	20.8
8	Chinstrap	Dream	53.5	19.9
9	Gentoo	Biscoe	53.4	15.8
10	Chinstrap	Dream	52.8	20.0
11	Chinstrap	Dream	52.7	19.8
12	Gentoo	Biscoe	52.5	15.6
13	Gentoo	Biscoe	52.2	17.1

Order by clause

```sql
SELECT species, island, bill_length_mm, bill_depth_mm
FROM penguins
ORDER BY bill_length_mm ASC
```

	species	island	bill_length_mm	bill_depth_mm
1	Adelie	Torgersen	NA	NA
2	Gentoo	Biscoe	NA	NA
3	Adelie	Dream	32.1	15.5
4	Adelie	Dream	33.1	16.1
5	Adelie	Torgersen	33.5	19.0
6	Adelie	Dream	34.0	17.1
7	Adelie	Torgersen	34.1	18.1
8	Adelie	Torgersen	34.4	18.4
9	Adelie	Biscoe	34.5	18.1
10	Adelie	Torgersen	34.6	21.1
11	Adelie	Torgersen	34.6	17.2
12	Adelie	Biscoe	35.0	17.9
13	Adelie	Biscoe	35.0	17.9

Order by clause

```sql
SELECT species, island, bill_length_mm, bill_depth_mm
FROM penguins
ORDER BY bill_length_mm, bill_depth_mm
```

	species	island	bill_length_mm	bill_depth_mm
1	Adelie	Torgersen	NA	NA
2	Gentoo	Biscoe	NA	NA
3	Adelie	Dream	32.1	15.5
4	Adelie	Dream	33.1	16.1
5	Adelie	Torgersen	33.5	19.0
6	Adelie	Dream	34.0	17.1
7	Adelie	Torgersen	34.1	18.1
8	Adelie	Torgersen	34.4	18.4
9	Adelie	Biscoe	34.5	18.1
10	Adelie	Torgersen	34.6	17.2
11	Adelie	Torgersen	34.6	21.1
12	Adelie	Biscoe	35.0	17.9
13	Adelie	Biscoe	35.0	17.9

group by clause

```sql
SELECT
  AVG(bill_length_mm) AS avg_bill_length,
  AVG(bill_depth_mm) AS avg_bill_depth
FROM penguins
```

	avg_bill_length	avg_bill_depth
1	43.92193	17.15117

group by clause

```sql
SELECT island,
  AVG(bill_length_mm) AS avg_bill_length,
  AVG(bill_depth_mm) AS avg_bill_depth
FROM penguins
GROUP BY island
```

	island	avg_bill_length	avg_bill_depth
1	Biscoe	45.25749	15.87485
2	Dream	44.16774	18.34435
3	Torgersen	38.95098	18.42941

group by clause

```sql
SELECT island, species,
  AVG(bill_length_mm) AS avg_bill_length,
  AVG(bill_depth_mm) AS avg_bill_depth
FROM penguins
GROUP BY island, species
```

	island	species	avg_bill_length	avg_bill_depth
1	Biscoe	Adelie	38.97500	18.37045
2	Biscoe	Gentoo	47.50488	14.98211
3	Dream	Adelie	38.50179	18.25179
4	Dream	Chinstrap	48.83382	18.42059
5	Torgersen	Adelie	38.95098	18.42941

group by clause - count

```sql
SELECT island, species,
  COUNT(*) AS count
FROM penguins
GROUP BY island, species
```

```
     island   species count
1    Biscoe    Adelie    44
2    Biscoe    Gentoo   124
3     Dream    Adelie    56
4     Dream Chinstrap    68
5 Torgersen    Adelie    52
```

group by clause - count

```sql
SELECT island, species,
  COUNT(*) AS count
FROM penguins
GROUP BY island, species
HAVING COUNT(*) < 100
```

```
     island   species count
1    Biscoe    Adelie    44
2     Dream    Adelie    56
3     Dream Chinstrap    68
4 Torgersen    Adelie    52
```

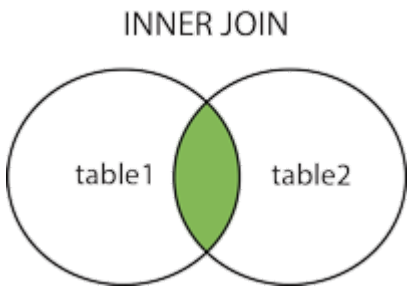
SQL Joins

How do we use multiple tables in a single query? Using JOIN clauses:

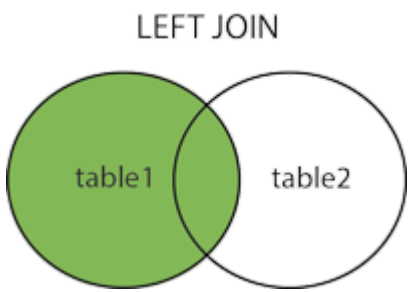
- `(INNER) JOIN`: Returns records that have matching values in both tables
- `LEFT (OUTER) JOIN`: Returns all records from the left table, and the matched records from the right table
- `RIGHT (OUTER) JOIN`: Returns all records from the right table, and the matched records from the left table
- `FULL (OUTER) JOIN`: Returns all records when there is a match in either left or right table

To learn more about joins (and practice), checkout the W3Schools website.

SQL Joins (cont. 1)

- 

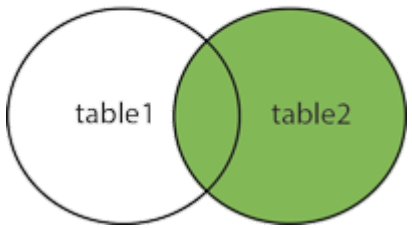

```
1 merge(x, y, all.x = FALSE, all.y = FALSE)
```

- 


```
1 merge(x, y, all.x = TRUE, all.y = FALSE)
```

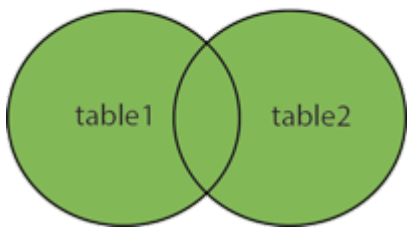
SQL Joins (cont. 2)

- RIGHT JOIN




```
1 merge(x, y, all.x = FALSE, all.y = TRUE)
```

- FULL OUTER JOIN




```
1 merge(x, y, all.x = TRUE, all.y = TRUE)
```

SQL Joins with CHS Data

We first start by loading the datasets from https://github.com/USCBiostats/data-science-data

```
# Download
chs_regional   <- read.csv("https://github.com/USCbiostats/data-science-dat
chs_individual <- read.csv("https://github.com/USCbiostats/data-science-dat

# Dropping some regions
chs_regional   <- subset(chs_regional, !townname %in% c("Alpine", "Lompoc")
chs_individual <- subset(chs_individual, townname != "Lancaster")
```

And we drop a few observations from both datasets so that they only have partially overlapping town information.

Data Intro: Regional

```
1 head(chs_regional)
```

```
       townname pm25_mass pm25_so4 pm25_no3 pm25_nh4 pm25_oc pm25_ec pm25_om
2 Lake Elsinore     12.35     1.90     2.98     1.36    3.64    0.62    4.36
3  Lake Gregory      7.66     1.07     2.07     0.91    2.46    0.40    2.96
4     Lancaster      8.50     0.91     1.87     0.78    4.43    0.55    5.32
6    Long Beach     19.12     3.23     6.22     2.57    5.21    1.36    6.25
7     Mira Loma     29.97     2.69    12.20     4.25   11.83    1.25   14.20
8     Riverside     22.39     2.43     8.66     3.14    5.27    0.94    6.32
  pm10_oc pm10_ec pm10_tc formic acetic  hcl hno3 o3_max o3106 o3_24   no2
2    4.66    0.63    5.29   1.18   3.56 0.46 2.63  66.70 54.42 32.23 17.03
3    3.16    0.41    3.57   0.66   2.36 0.28 2.28  84.44 67.01 57.76  7.62
4    5.68    0.56    8.61   0.88   2.88 0.22 1.80  54.81 43.88 32.86 15.77
6    6.68    1.39    8.07   1.57   2.94 0.73 2.67  39.44 28.22 18.22 33.11
7   15.16    1.28   16.44   1.90   5.14 0.46 3.33  70.65 55.81 26.81 19.62
8    6.75    0.96    7.72   1.72   3.92 0.47 3.43  77.90 62.04 31.50 19.95
   pm10 no_24hr pm2_5_fr iacid oacid total_acids       lon      lat
2 34.25    7.07    14.53  3.09  4.74        7.37 -117.3273 33.66808
3 20.05      NA     9.01  2.56  3.02        5.30 -117.2752 34.24290
4 25.04   12.68       NA  2.02  3.76        5.56 -118.1542 34.68678
6 38.41   36.76    22.23  3.40  4.51        7.18 -118.1937 33.77005
7 70.39   26.90    31.55  3.79  7.04       10.37 -117.5159 33.98454
```

```
1 dim(chs_regional)
```

```
[1] 10 27
```

Data Intro: Regional (cont.)

```
1 unique(chs_regional$townname)
```

```
[1] "Lake Elsinore" "Lake Gregory"  "Lancaster"     "Long Beach"
[5] "Mira Loma"     "Riverside"     "San Dimas"     "Atascadero"
[9] "Santa Maria"   "Upland"
```

Data Intro: Individual

```
1 head(chs_individual)
```

```
   sid  townname male race hispanic    agepft height weight      bmi asthma
91 176 San Dimas    0    W        0  9.954825    140     69 16.00186      0
92 178 San Dimas    1    A        0  9.744011    138     78 18.61717      1
93 179 San Dimas    1    M        1 10.496920    143     93 20.67227      0
94 181 San Dimas    1    M        0 10.275154    143    119 26.45162      1
95 183 San Dimas    0    W        1  9.804244    141     79 18.06201      1
96 184 San Dimas    0    O        1 11.008898    144     72 15.78283      0
   active_asthma father_asthma mother_asthma wheeze hayfever allergy
91             0             0             1      0       NA       0
92             1             1             0      1        1       0
93             0             0             0      0        0       0
94             1             0             1      1        1       1
95             1             0             0      1        0       1
96             0             0             0      0        0       0
   educ_parent smoke pets gasstove      fev      fvc      mmef
91           5     0    1        1 2130.232 2377.698 2411.0302
92           4     1    1        0 1658.054 2075.839  869.7987
93           3     0    1        1 2187.198 2566.509 2310.3511
94           3     0    1        1 2206.000 2501.000 2659.0000
95           3     0    1        1 2178.331 2598.037 2267.0012
```

```
1 dim(chs_individual)
```

```
[1] 1100   23
```

Data Intro: Individual (cont.)

```
1 unique(chs_individual$townname)
```

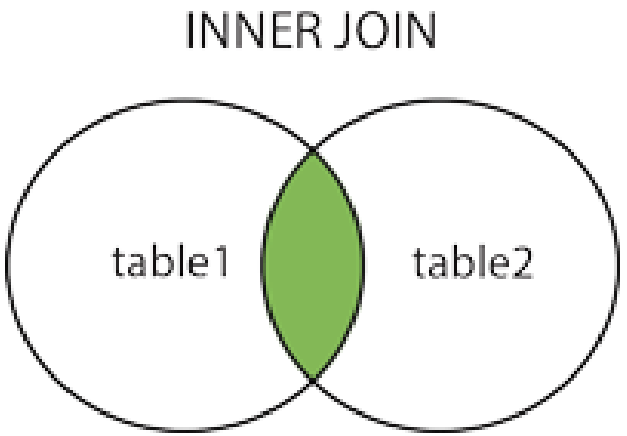
```
[1] "San Dimas"     "Atascadero"    "Riverside"     "Mira Loma"
[5] "Alpine"        "Lake Elsinore" "Lake Gregory"  "Long Beach"
[9] "Santa Maria"   "Upland"        "Lompoc"
```

Set up SQL database

```
# Set up the dataset
library(RSQLite)
con <- dbConnect(SQLite(), ":memory:")
dbWriteTable(con, "regional", chs_regional)
dbWriteTable(con, "individual", chs_individual)
```

We now can illustrate the joins.

SQL INNER join

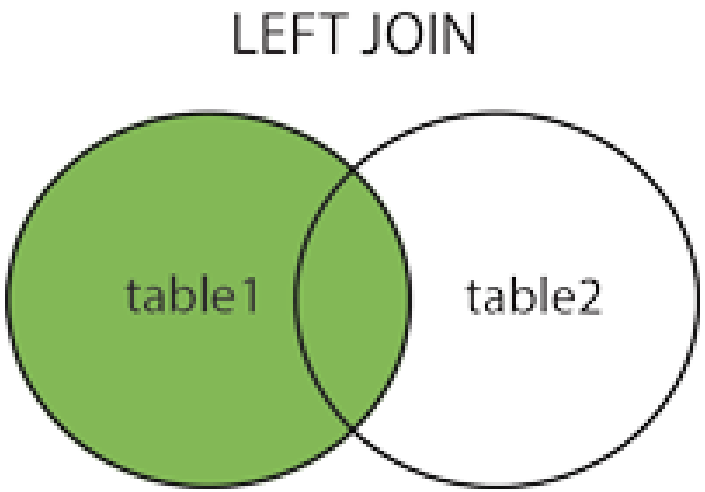


- `chs_individual` has 1100 rows
- `chs_regional` has 10 rows

```
ans <- dbGetQuery(con,
  "SELECT *
  FROM regional AS a INNER JOIN individual AS b
  ON a.townname = b.townname
  ")
nrow(ans)
```

```
[1] 900
```

SQL LEFT join

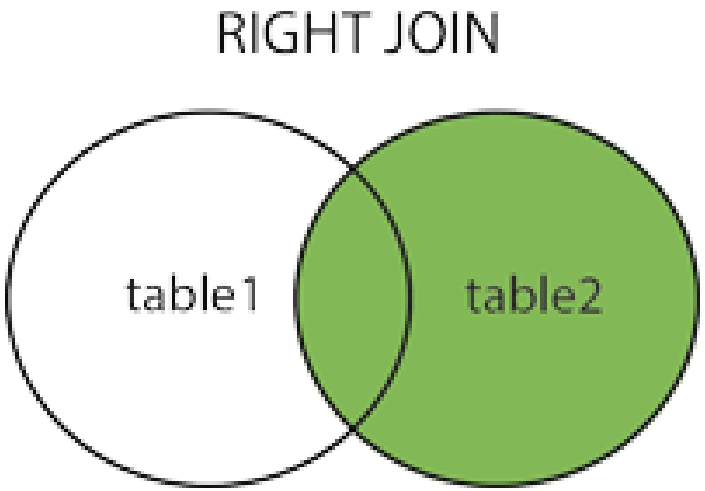


- `chs_individual` has 1100 rows
- `chs_regional` has 10 rows

```
1 ans <- dbGetQuery(con,
2   "SELECT *
3   FROM regional AS a LEFT JOIN individual AS b
4   ON a.townname = b.townname")
5 nrow(ans)
```

```
[1] 901
```

SQL RIGHT join

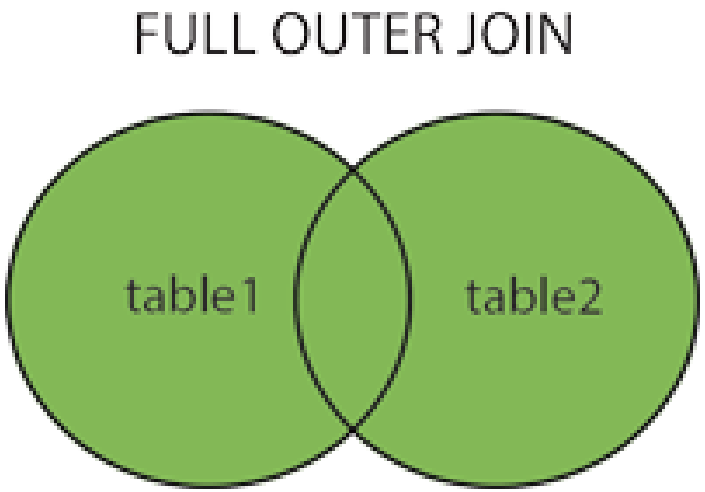


- `chs_individual` has 1100 rows
- `chs_regional` has 10 rows

```
ans <- dbGetQuery(con,
  "SELECT * /* RIGHT joins are not supported by SQLite*/
  FROM individual AS a LEFT JOIN regional AS b
  ON a.townname = b.townname")
nrow(ans)
```

```
[1] 1100
```

SQL FULL join



- `chs_individual` has 1100 rows
- `chs_regional` has 10 rows

```
ans <- dbGetQuery(con,
  "SELECT *
  FROM individual AS a FULL JOIN regional AS b
  ON a.townname = b.townname")
nrow(ans)
```

```
[1] 1101
```

Disconnect database

```
1 dbDisconnect(con)
```

What happens when there are no records matching

Step 1: Let's break things

```
chs_individual2 <- subset(
  chs_individual, !(townname %in% chs_regional$townname)
)

unique(chs_individual2$townname)
```

```
[1] "Alpine" "Lompoc"
```

```
chs_regional$townname
```

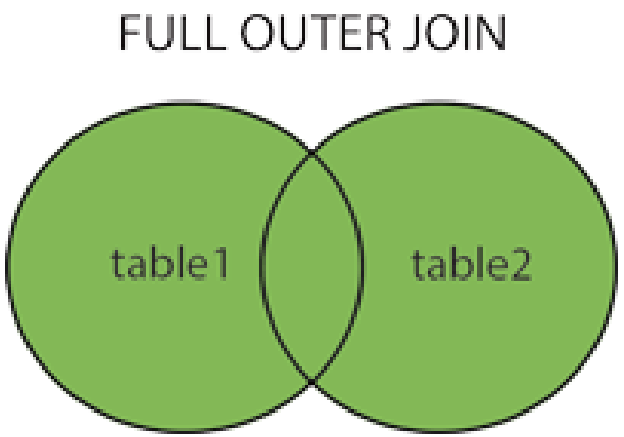
```
 [1] "Lake Elsinore" "Lake Gregory"  "Lancaster"     "Long Beach"
 [5] "Mira Loma"     "Riverside"     "San Dimas"     "Atascadero"
 [9] "Santa Maria"   "Upland"
```

Step 2: Rebuild the Database

```
con <- dbConnect(SQLite(), ":memory:")
dbWriteTable(con, "regional", chs_regional)
dbWriteTable(con, "individual", chs_individual2)
```

What happens now?

Re-running the full join:



- `chs_individual` has 200 rows
- `chs_regional` has 10 rows

```
1 ans <- dbGetQuery(con,
2   "SELECT *
3   FROM individual AS a FULL JOIN regional AS b
4   ON a.townname = b.townname")
5 nrow(ans)
```

```
[1] 210
```

```
1 dbDisconnect(con)
```

What happens when we have multiple matching records in both tables?

```
con <- dbConnect(SQLite(), ":memory:")
dbWriteTable(con, "regional_dpl", rbind(chs_regional, chs_regional))

nrow(rbind(chs_regional, chs_regional))
```

```
[1] 20
```

Joins with duplicates

```
ans <- dbGetQuery(con,
  "SELECT *
  FROM regional_dpl AS a FULL JOIN regional_dpl AS b
  ON a.townname = b.townname")
nrow(ans)
```

```
[1] 40
```

```
ans <- dbGetQuery(con,
  "SELECT * FROM regional_dpl AS a INNER JOIN regional_dpl AS b
    ON a.townname = b.townname
  ")
nrow(ans)
```

```
[1] 40
```

Distinct entries

```
ans <- dbGetQuery(con,
  "SELECT DISTINCT * FROM regional_dpl AS a INNER JOIN regional_dpl AS b
    ON a.townname = b.townname
  ")
nrow(ans)
```

```
[1] 10
```

```
dbDisconnect(con)
```